Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

2. Conditional frequency distribution using the Brown Corpus

Aim: To find the frequency of selected words in selected genres of the Brown Corpus using NLTK's `ConditionalFreqDist`.

```
import nltk
from nltk.corpus import brown
from nltk.probability import ConditionalFreqDist

# Download the Brown Corpus if it is not already installed.
nltk.download('brown')

# Select the genres to compare.
genres = ['news', 'fiction', 'romance']

# Select the words whose frequencies we want to examine.
selected_words = ['the', 'love', 'government', 'man', 'woman']

# Create a list of (genre, word) pairs.
genre_word_pairs = []

for genre in genres:
    # Read the words belonging to the current genre.
    words = brown.words(categories=genre)

    # Convert words to lowercase for case-insensitive counting.
    words = [word.lower() for word in words]

    # Add each selected word together with its genre.
    for word in words:
        if word in selected_words:
            genre_word_pairs.append((genre, word))

# Build the conditional frequency distribution.
cfd = ConditionalFreqDist(genre_word_pairs)

# Display the frequency of every selected word in each genre.
for genre in genres:
    print("\nGenre:", genre)

    for word in selected_words:
        print(word, ":", cfd[genre][word])
```

Sample output

The following is an example of the output format; the actual counts are generated by NLTK.

```
Genre: news
the : [count]
love : [count]
government : [count]
man : [count]
woman : [count]
```

```
Genre: fiction
the : [count]
love : [count]
government : [count]
man : [count]
woman : [count]

Genre: romance
the : [count]
love : [count]
government : [count]
man : [count]
woman : [count]
```

Explanation:

• `brown.words(categories=genre)` retrieves the words belonging to a genre.

• `ConditionalFreqDist` maintains a separate frequency distribution for each condition—in this case, the genre.

• `cfd['romance']['love']` returns the frequency of the word `love` in the romance genre.